Comparative Analysis of Recurrent Neural Networks and Logistic Regression Models in Stock Price Prediction

To what extent are Recurrent Neural Networks more effective than traditional Logistic Regression models in predicting stock price movements?

Computer Science
Machine Learning

Word Count: 3993

Table of Contents

1 Introduction

The stock market is one of the places where the technological and informational edge bring great profit returns on investment. That is why, it attracts many large investment firms and professional traders who utilise large resources to make predictions on the price movements. The prices on the market are dependent on many factors, including but not limited to government policies, natural disasters, market sentiment, micro and macroeconomic factors.[1] Traditional methods for capturing signals from markets include the use of linear classifiers and regression models. Logistic regression is one of those traditional methods that has been used in finance since the 1980s.[2]

Over the years, advancements in machine learning gave birth to new complex mathematical models, specifically neural networks or deep learning models, whose use in quantitative finance was proved to be more lucrative than traditional prediction methods. Those include Convolutional Neural Networks, Generative Adversarial Networks, Transformer Models, and Recurrent Neural Networks. Among them, Recurrent Neural Networks are the ones designed for sequential data such as stock data and have the ability to capture long-term dependencies in the market.[3]

Despite one's complexity over the other, the research on superiority of models in predicting stock price movements is limited. On one hand, findings of Fischer, T. & Krauss, C. (2017) suggest that their Long Short-Term Memory model, Recurrent Neural Network, outperformed traditional linear models, including Logistic Regression, in predicting directions. On the other hand, findings of Makridakis S, Spiliotis E, Assimakopoulos V

---

[1] Kenton, Will. "What Are the Key Factors That Cause the Market to Go Up and Down?" *Investopedia*, Dotdash Meredith, 21 Sept. 2023, www.investopedia.com/ask/answers/100314/what-are-key-factors-cause-market-go-and-down.asp.

[2] Zaidi, Makram, and Amina Amirat. "Forecasting Stock Market Trends by Logistic Regression and Neural Networks: Evidence from KSA Stock Market." *International Journal of Economics, Commerce and Management*, vol. 4, no. 6, June 2016, pp. 220–234. ijecm.co.uk/wp-content/uploads/2016/06/4614.pdf .

[3] Pandey, Rajat. *Stock Price Prediction: An Integrated Review of Traditional and Modern Computational Models. SSRN*, 2025, https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5353725.

(2018) argue that classical statistical models show similar or better results than deep ML models and reduce computational cost. Therefore, this extended essay aims to define the effectiveness of both models in predicting stock price movements by answering the research question:

"To what extent are Recurrent Neural Networks more effective than traditional Logistic Regression models in predicting stock price movements?"

To do so, this essay will first introduce the theoretical background, conduct an experiment, then analyze the results to draw a conclusion.

2 Technical Foundations

All theoretical foundations of RNNs and Logistic regression models related to the research question are given in this section.

2.1 Time Series and Stationarity

A **time series** is a sequence of data points measured at regular time intervals, for instance, daily stock prices or annual revenue records.[4]

Time series data can be both *stationary* and *non-stationary*, but as of this topic, the stock data is often *non-stationary,* whereby its *means, variances,* and *covariances* change over time due to *trends, cycles,* or *random walks*.[5] This type of data leads to false and unreliable relationships between variables if not transformed to *stationary data*.[6]

2.2 Recurrent neural networks (RNNs)

RNNs are deep learning models used to model sequential data. Unlike standard neural networks, where inputs and outputs are independent of each other, RNNs have a *feedback mechanism*, which uses hidden layers to allow data to remember specific information about a sequence.[7]

RNNs generally perform well on short sequences, but they have a major limitation for long sequences due to limited memory - *the vanishing gradient problem,* repeated multiplication of gradients in *backpropagation* through time.[8] So simple RNNs would not be

---

[4] "Time Series Analysis and Forecasting." *GeeksforGeeks*, 23 July 2025, www.geeksforgeeks.org/machine-learning/time-series-analysis-and-forecasting/#what-is-time-series-forecasting.
[5] "Introduction to Non-Stationary Processes." *Investopedia*, 5 Jan. 2022, www.investopedia.com/articles/trading/07/stationary.asp.
[6] "Introduction to Non-Stationary Processes." *Investopedia*, 5 Jan. 2022, www.investopedia.com/articles/trading/07/stationary.asp.
[7] Kalita, Debasish. "A Brief Overview of Recurrent Neural Networks (RNN)." *Analytics Vidhya*, 7 Nov. 2023, www.analyticsvidhya.com/blog/2022/03/a-brief-overview-of-recurrent-neural-networks-rnn/.
[8] "RNN vs LSTM vs GRU vs Transformers." *GeeksforGeeks*, 23 July 2025, www.geeksforgeeks.org/deep-learning/rnn-vs-lstm-vs-gru-vs-transformers/.

effective for the time series forecasting experiment of this essay, but LSTM, an improved version of RNNs, would be better choice due its superior performance on *high-complexity sequences* over other types of RNNs like GRUs (Gated Recurrent Units).[9]

2.2.1 Long Short-Term Memory (LSTM) neural networks

LSTM, an enhanced type of RNNs with modified architecture, is developed to effectively address the challenges of modeling long sequences encountered in conventional RNNs by adding *memory cells* and *gates* that hold information for extended time periods.[10]

2.2.1.1 Architecture

In this subsection, the architecture of LSTMs will be explained thoroughly.

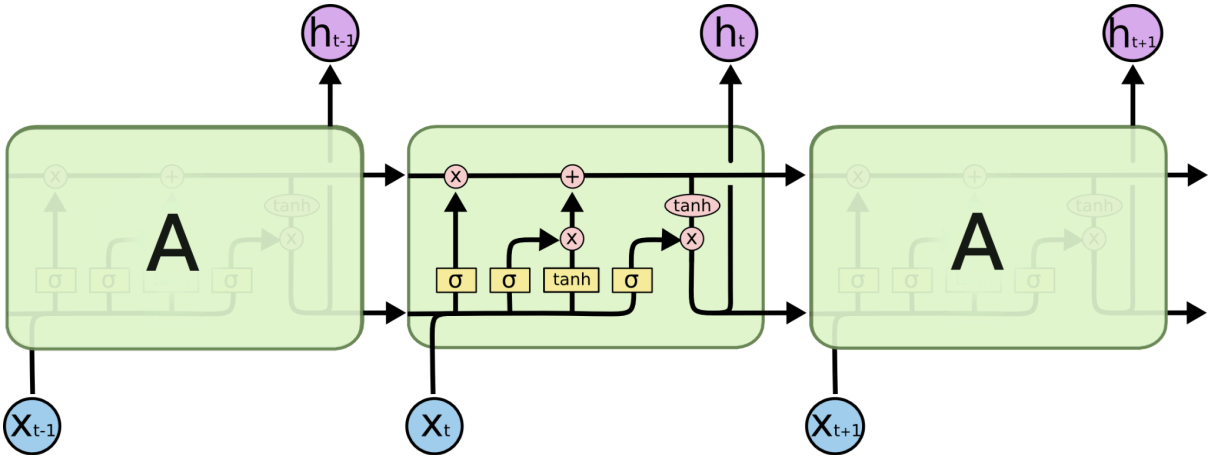


Figure 1: Architecture of LSTM (Source: *Understanding LSTM Networks - Colah's Blog*)

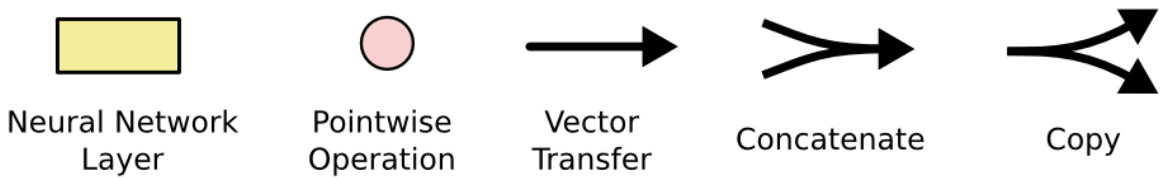


Figure 2: Notation used in Fig 1 and upcoming figures (Source: *Understanding LSTM Networks - Colah's Blog*)

---

[9] Cahuantzi, Roberto, Xinye Chen, and Stefan Güttel. "A Comparison of LSTM and GRU Networks for Learning Symbolic Sequences." *arXiv*, 5 July 2021, arxiv.org/abs/2107.02248 .

[10] "What is LSTM - Long Short Term Memory?" *GeeksforGeeks*, 26 July 2025, www.geeksforgeeks.org/deep-learning/deep-learning-introduction-to-long-short-term-memory/.

In Figure 1, the *repeating* module of the LSTM network that models sequential inputs over time steps $t-1$, t, and $t+1$ is given. Here, each module has the current input x_t and the hidden state h_{t-1}, and gives the output of a newly updated hidden state h_t and the cell state to carry long-term information. This flow of data is controlled via *pointwise operations* (Figure 2) : X for gating and $+$ for combining. Similar to RNNs, LSTMs also have a chain structure, but instead of one, LSTMs have **four interactive neural network layers** as shown in Figure 3.[11]

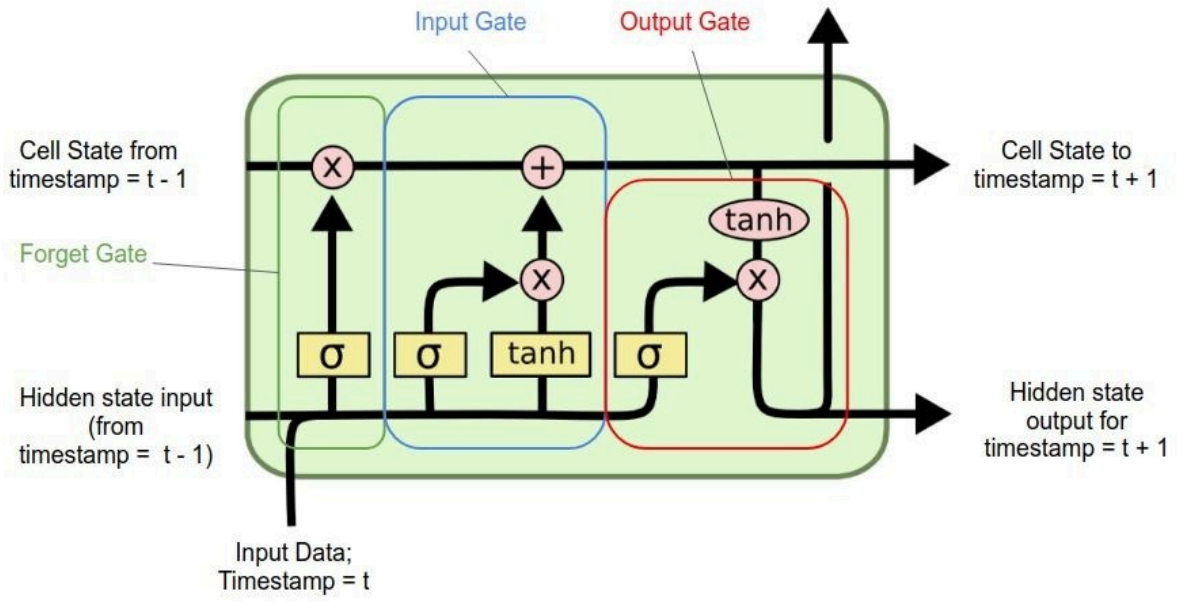


Figure 3: Schematic Diagram of LSTM Representing Gates (Source: *LSTM networks - Abdulllah Al Rahman*)

The key part of this LSTM architecture is the ***cell state*** as shown in Figure 4. This cell state is where the information flows down the entire chain. LSTM has the ability to add or

[11]Olah, Christopher. "Understanding LSTM Networks." *Colah's Blog*, 27 Aug. 2015, colah.github.io/posts/2015-08-Understanding-LSTMs/ .

remove information to the cell state by using its regulated *gates* mentioned above.[12]

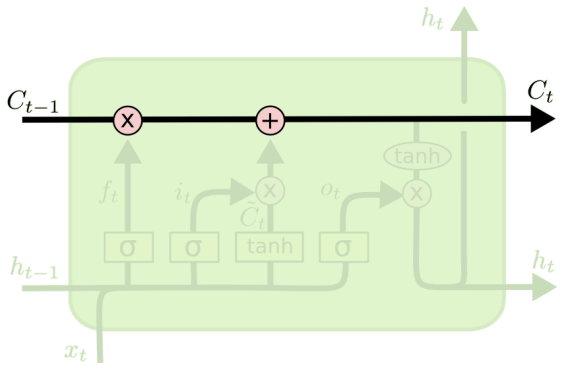


Figure 4: The ***cell state*** **-** horizontal line running through the top of the diagram (Source: *Understanding LSTM Networks - Colah's Blog*)

In the next section, the way these components interact with the cell state to update and output LSTM's memory will be mathematically explained.

2.2.1.2 Operation

The **Forget Gate**, in Figure 5, is the first layer in LSTM operation that decides whether to use information from the previous module depending on the type of prediction.[13]

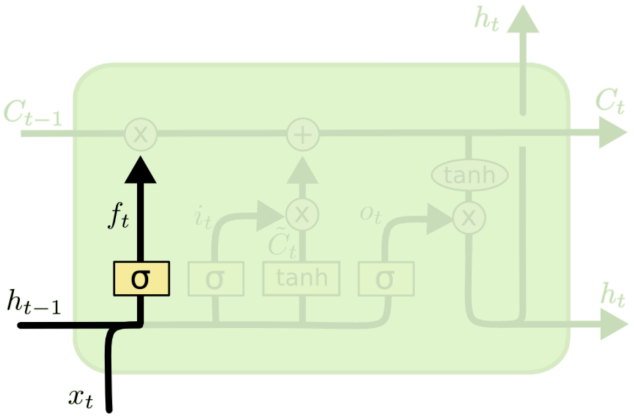


Figure 5: Schematic Diagram of LSTM Forget Gate (Source: *Understanding LSTM Networks - Colah's Blog*)

---

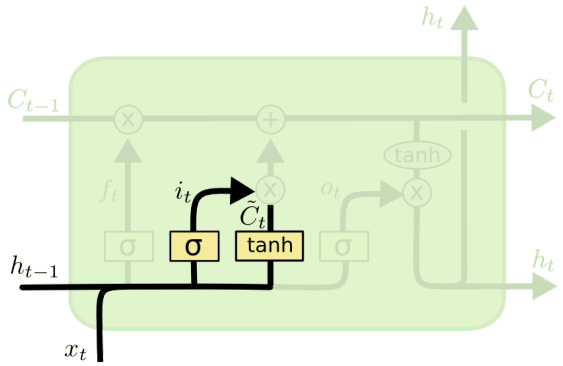
[12] Olah, Christopher. "Understanding LSTM Networks." *Colah's Blog*, 27 Aug. 2015, colah.github.io/posts/2015-08-Understanding-LSTMs/ .
[13] Olah, Christopher. "Understanding LSTM Networks." *Colah's Blog*, 27 Aug. 2015, colah.github.io/posts/2015-08-Understanding-LSTMs/ .

The sigmoid function, that gives any real values between 0 and 1, in scale that 0 means getting rid of information and 1 means keeping it, used in *forget gate layer* is given in Figure 6, where the $x^{(t)}$ is the current input and $h^{(t-1)}$ is the previous hidden layer vector:

$$f_i^{(t)} = \sigma\left(b_i^f + \sum_j U_{i,j}^f \, x_j^{(t)} + \sum_j W_{i,j}^f \, h_j^{(t-1)}\right)$$

Figure 6: Sigmoid Function of LSTM Forget Gate (Source*: Deep Learning (Goodfellow, Bengio, & Courville, 2016), Eq. 10.40*)

The **Input Gate**, as shown in Figure 7, is the next layer that decides which new information should be stored in the cell state by processing the combination of the current input $x^{(t)}$ and is the previous hidden layer vector $h^{(t-1)}$.[14]

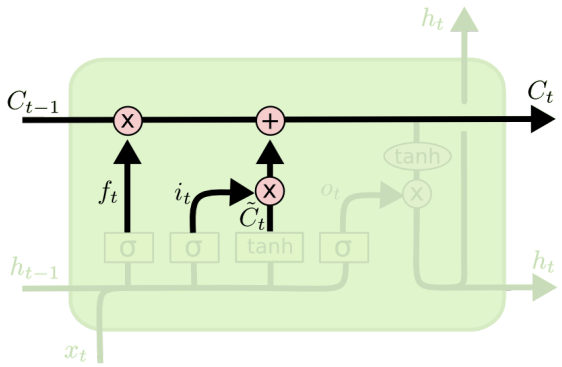


$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

Figure 7: Schematic Diagram of LSTM Input Gate (Source: *Understanding LSTM Networks - Colah's Blog*)

In Figure 8, the **Cell State Update** step is given, in which the actions decided to be taken in the *forget* and *input gate* are applied.[15]

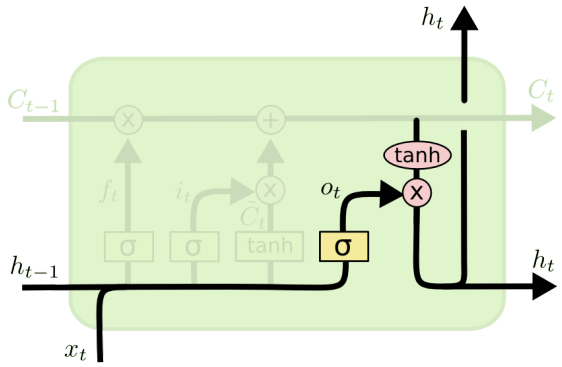
[14] Olah, Christopher. "Understanding LSTM Networks." *Colah's Blog*, 27 Aug. 2015, colah.github.io/posts/2015-08-Understanding-LSTMs/ .
[15] Olah, Christopher. "Understanding LSTM Networks." *Colah's Blog*, 27 Aug. 2015, colah.github.io/posts/2015-08-Understanding-LSTMs/ .



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Figure 8: Schematic Diagram of LSTM Cell Update (Source: *Understanding LSTM Networks - Colah's Blog*)

After the cell state gets updated, LSTM outputs by its **Output Gate**, as shown in Figure 9. To determine the next hidden state h_t, It first uses *sigmoid layer* that decides what parts of the cell state need to be output and *tanh* layer to transform the values between -1 and 1.[16]



$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$
$$h_t = o_t * \tanh\left(C_t\right)$$

Figure 9: Schematic Diagram of LSTM Output Gate Layer (Source: *Understanding LSTM Networks - Colah's Blog*)

2.2.2 Applying LSTM to Time Series Prediction

As mentioned above, LSTMs, by their *gating layers,* can identify patterns over long time horizons and remove *noise,* befitting for time series modelling. This application consists

[16]Olah, Christopher. "Understanding LSTM Networks." *Colah's Blog*, 27 Aug. 2015, colah.github.io/posts/2015-08-Understanding-LSTMs/ .

of structuring sequential input data and defining model's *parameters* and tuning *hyperparameters*.

2.2.2.1 Parameters and Hyperparameters of LSTM

In machine learning, including LSTM models, the terms *parameters* and *hyperparameters* are two different but related concepts. Model parameters, *weights* and *biases* for LSTMs, are variables of the selected model that **can be learned by the model** based on the given *data* and *hyperparameters*.[17]

A model hyperparameter is a top-level parameter whose value is fixed by **the model designer** before the training. They can not be learned during training, thus remaining unchanged by the end of the learning process.[18] The *hyperparameters* for neural networks include number of hidden layers, number of neurons per layer, loss function, batch size, and etc.[19] Optimization of the *hyperparameters* is **necessary** for better performance.

2.2.2.2 Optimizing LSTM Hyperparameters

The *hyperparameters* are chosen based on computational resources and desired accuracy results. There are many hyperparameter optimization methods such as *grid* or *random search*, *Bayesian optimization*, *Hyperband*, and *racing*.[20]

Another alternative efficient approach is using the *KerasTuner* library in Python that comes with Bayesian Optimization, Hyperband, and Random Search algorithms.[21]

---

[17] "Difference Between Model Parameters vs Hyperparameters." *GeeksforGeeks*, 2025, www.geeksforgeeks.org/machine-learning/difference-between-model-parameters-vs-hyperparameters/.
[18] "Difference Between Model Parameters vs Hyperparameters." *GeeksforGeeks*, 2025, www.geeksforgeeks.org/machine-learning/difference-between-model-parameters-vs-hyperparameters/.
[19] Nyuytiymbiy, Kizito. "Parameters, Hyperparameters, Machine Learning." *Towards Data Science*, 7 Mar. 2023, towardsdatascience.com/parameters-and-hyperparameters-aa609601a9ac .
[20] Bischl, Bernd, et al. "Hyperparameter Optimization: Foundations, Algorithms, Best Practices and Open Challenges." *arXiv*, 24 Nov. 2021, doi.org/10.48550/arXiv.2107.05847 .
[21] O'Malley, Tom, et al. *KerasTuner*. *Keras*, 2019, https://keras.io/keras_tuner/.

2.2.3 Advantages and disadvantages

Ultimately, based on its technical background, the advantages and disadvantages of LSTMs on financial time series are evaluated below.

One one hand, LSTMs are effective at identifying dependencies in long time series sequences without the *vanishing gradient problem* faced in simple RNNs. Secondly, through sophisticated gating structures, LSTMs can model *non-linear relationships*, mostly multi-day or seasonal market patterns, which makes stock forecasting easier without manual feature engineering.

On the other hand, LSTMs in financial time series are somewhat limited. Firstly, training LSTMs with large datasets needs to have *substantial computational resources*. Secondly, they are apt to *overfitting*, especially when data is limited or very noisy. Lastly, their *black-box* structure, meaning less interpretability, causes limitations in its application in finance as legally models must provide auditable interfaces.[22]

[22]Das, Saurabh, et al. "Trustworthy Artificial Intelligence in Finance: Explainability, Fairness, and Related Principles for AI in Financial Services." *arXiv*, 16 May 2025, arxiv.org/abs/2505.24650 .

2.3 Logistic Regression (LR)

Logistic regression is a machine learning model that predicts the probability of binary outcome in **classification** problems. LR is a **supervised learning** model, which means it needs *labeled* data that has target output for its every corresponding input. The relationship learned from this data is used to model the outcome for the unseen data.[23]

2.3.1 Sigmoid Activation Function & Decision Boundary

The whole mechanism of LRs rely on the *sigmoid function* which outputs probability values between 0 and 1. Mathematical equation of this function is given in Figure 10:

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

Figure 10: Sigmoid/Logistic Function of LRs (Source*: Deep Learning (Goodfellow, Bengio, & Courville, 2016), Eq. 3.30*)

The graph of the *sigmoid function* is given below:

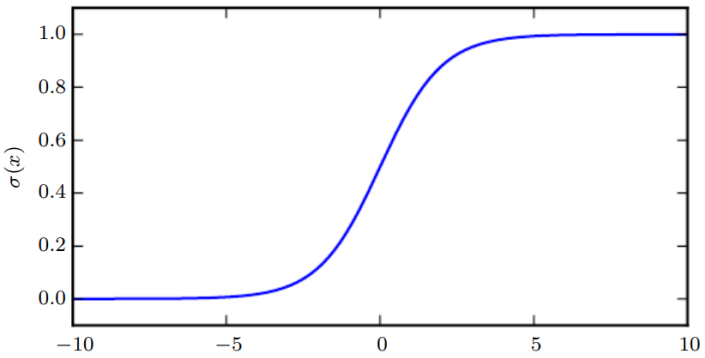


Figure 11: The graph of LR sigmoid function (Source*: Deep Learning (Goodfellow, Bengio, & Courville, 2016), Fig. 3.3*)

As shown in Figure 11, when its argument is very positive or very negative, the sigmoid function becomes very flat and insensitive to small changes in its input.[24] But to

[23] "Logistic Regression in Machine Learning." *GeeksforGeeks*, 2 Aug. 2025, www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/.
[24] Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

make a binary decision, the output needs to be classified as either 0 or 1. In logistic regression, usually a threshold value of 0.5 is used to identify the class, which is called *decision boundary* - a margin that separates data into different classes according to their predictions. That is, if the probability value is below than 0.5, the decision is no, else yes.[25]

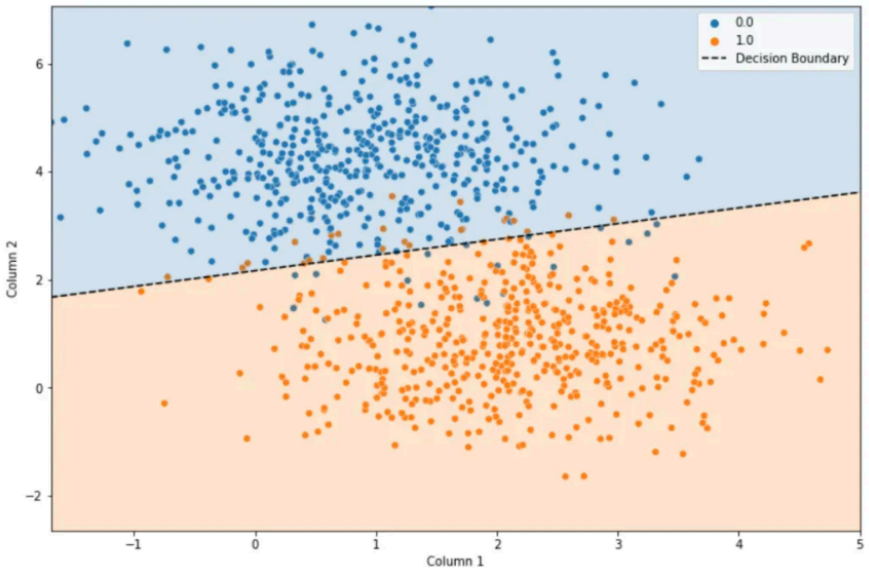


Figure 12: The plot of LR decision boundary (Source: *Logistic Regression in Machine Learning - Chitwan Manchanda*)

In the plot given in Figure 12, it is shown that LR can separate the two classes almost perfectly.[26]

2.3.2 Application of Logistic Regression for Time Series Prediction

Due to its low computational requirements for training and accessibility, the application of LR models to time series forecasting is widely researched. But as time series data is usually *non-stationary* (explained in 3.2), it requires *data and feature engineering*.

---

[25] "Logistic Regression in Machine Learning." *GeeksforGeeks*, 2 Aug. 2025, www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/.

[26] Manchanda, Chitwan. "Logistic Regression in Machine Learning (from Scratch!!)." *Medium*, 11 Mar. 2022, medium.com/aiguys/logistic-regression-in-machine-learning-from-scratch-62f45048c571 .

2.3.2.1 Data and Feature Engineering

One of the most famous approaches is to build *lagged returns* which use past returns to predict the future. For instance, let's assume r_t to be the return on day t. Then depending on the stock type and prediction horizon, the amount of lag n is decided. So, r_t is used to predict r_{t+n}. This approach makes sure short-term changes are also learning subsequent direction.

Another approach would be using *moving averages*, also known as *rolling averages*, which reduces the noiseness of time series data to reveal underlying *trends* and *cycles*. It is calculated using sequential segments of data points, whose number is called *window length*, to get a series of averages.[27]

There are other complex technical indicators for prediction optimization such as *Relative Strength Index(RSI), Bollinger Bands, Intraday Momentum Index (IMI)* and etc. The complexity of these indicators is in their construction as it requires expertise and additional market assumptions. So they will not be used in this essay.

2.3.2.2 Overfitting and Regularization

The application of LR models to time series can cause a common problem called *overfitting*. It happens for LR models when they become so complex because of many engineered features or the non-linear orientation between the features and the target. *Overfitting* causes low accuracy on the *test set*, meaning low predictive power on the new data.[28]

A technique to avoid *overfitting* is called *regularization*. It avoids it by adding a *penalty term* to the objective function, which reduces the complexity of the model and solves

[27]Frost, Jim. "Using Moving Averages to Smooth Time Series Data." *Statistics by Jim*, statisticsbyjim.com/time-series/moving-averages-smoothing/ .
[28] RITHP. "Logistic Regression and Regularization: Avoiding Overfitting and Improving Generalization." *Medium*, 5 Jan. 2023,medium.com/@rithpansanga/logistic-regression-and-regularization-avoiding-overfitting-and-improving-generalization-e9afdcddd09d .

overfitting. The *penalty term* is a *hyperparameter* (explained in subsection 2.2.2.1): the higher *penalty term*, the stronger regularization and a simpler model and vice versa.[29] *Regularization* is **very necessary** in the application of LR models to time series. Since LR models are inherently not suitable for financial prediction, features like *lagged returns* and *moving averages* are added, which causes *overfitting.* For LSTM, which is suitable for time series data, the risk of overfitting is lower than that of LRs.

2.3.3 Advantages and disadvantages

Ultimately, based on its technical background, the advantages and disadvantages of LR models on financial time series are evaluated below.

On one hand, LR models are one of the most *interpretable* classification algorithms, which makes its implementation way easier. Secondly, they require *less computational resources* for training, which means multiple stocks with large datasets can be trained at the same time with less resources. Lastly, if applied properly, *feature engineering* can help to identify valuable *signals* from the market in the *short-term.*

On the other hand, the effectiveness of LR models in financial time series is somewhat limited. Firstly, even with manual *data and feature engineering*, their linear *decision boundary* limits their ability to identify complex patterns in stock markets. Lastly, LR models treat the parts of price movements *independently*, which is a remarkable disadvantage as time series data is *sequential* data.

[29]RITHP. "Logistic Regression and Regularization: Avoiding Overfitting and Improving Generalization." *Medium*, 5 Jan. 2023,medium.com/@rithpansanga/logistic-regression-and-regularization-avoiding-overfitting-and-improving-generalization-e9afdcddd09d .

3 Methodology and Experimental Setup

The experiment of this study is done in Python 3.12 using *scikit-learn* library for LR; *TensorFlow* with *Keras* for LSTM; and *Pandas* and *Numpy* for data handling. This study followed the steps below to answer the RQ.

3.1 Step 1: Data Collection

First, financial data was attempted to be collected from Yahoo Finance[30], using the *yfinance*[31] library, as this library provides *Adjusted Close Prices* which are the optimized data for time series forecasting, as they also include post-market action. But extensive *rate limits* from *yfinance* made the study rely on another source for stock market API - *Alpha Vantage*[32]. The API from this source provided 20-year historical stock price data for the companies given in Table 1:

Ticker Symbol	Company Name	Sector	Market Cap(Approx.)
MSFT	Microsoft Corp.	Technology	$3.0T+
AMBA	Ambarella Inc.	Technology	~$2B
AMZN	Amazon.com Inc.	E-commerce	$2.0T+
SFTX	Shift Technologies Inc.	E-commerce	<$500M (micro cap)
JNJ	Johnson & Johnson	Healthcare	~$350B
BLUE	Bluebird bio Inc.	Healthcare	<$1B
JPM	JPMorgan Chase & Co.	Finance	~$600B
LC	LendingClub Corp.	Finance	<$1B

[30] "Yahoo Finance - Stock Market Live, Quotes, Business and Finance News." *Yahoo Finance*, finance.yahoo.com .
[31] "Yfinance." *PyPI*, 21 Jan. 2024, pypi.org/project/yfinance.
[32] "API Documentation." *Alpha Vantage*, www.alphavantage.co/documentation/.

XOM	Exxon Mobil Corp.	Oil & Gas	~$500B
SM	SM Energy Co.	Oil & Gas	~$6B

Table 1 : List of stocks chosen for the study

The companies in Table 1 are chosen intentionally, 5 **different sectors** with each having one **large cap** company and one **small/mid cap** company.

3.2 Step 2: Data Cleaning & Preprocessing

3.2.1 Data cleaning

Historical 4-year financial stock data from January 1, 2021 to December 31, 2024 was extracted from the CSV files provided by *Alpha Vantage*, as it is free from the remarkable events such as Covid pandemic and election of Donald Trump and also includes the growth in overall market due to emergence of AI, which created enough growth for comprehensive analysis.

Another limitation was that the free API key from this source only provided unadjusted *OHLCV* values which need to be optimized as just taking *close* values is not enough for forecasting; it opens room for *outliers* in data. So an *outlier filter* of $\pm$25% was used to guard from outliers, especially happening in large-cap companies (AMZN's sudden price level jumps in 2022).

After these changes all data was saved to new CSV files that only contain *datetime* (2021-2024) and custom *adj_close* prices.

3.2.2 Splitting the Data into Training and Testing Sets

The preprocessed data was split into training and testing data sets using a split ratio of 80:20, widely accepted convention.

3.3 Step 3: Model Implementation

The *training data* was fitted to both models, and their evaluation was made by the *testing data*.

3.3.1 Data and Feature Engineering for LR

As explained in subsection 2.3.2.1, the most optimal features to add to LR are *lagged returns* and *moving averages*, and they were added to LR during the model implementation: lags of returns (1-5), short MAs (5,10), MA ratios, 10-day volatility.

3.3.2 Hyperparameters Tuning for LSTM

As explained in subsection 2.2.2.2, tuning LSTM with complex hyperparameters requires so much expertise and resources. But, the purpose of this study is not to fully optimize the model, but to have a fair comparison. That is why, in this study the *KerasTuner* library in Python, with its built-in hyperparameter optimization with several benefits, was employed to optimize the LSTM model. *Sequence length* of 60, 60-day window, was chosen to balance signal capture, as too short or too long causes under/over-smoothing. Moreover, to prevent overfitting, single recurrent layer with 50 units was chosen as it is a standard baseline for time series. Additionally, *batch size* of 32 was chosen to balance noise in estimation, because smaller batch size would cause unstable updates whereas larger batch size would cause over-smoothed gradient.

3.4 Step 4: Prediction

The time horizon for the stock price prediction varies depending on the objective. This study employs **1-day ahead** prediction as error metrics such as MSE and MAE are calculated to be the lowest and increase exponentially for 10-day and 100-day horizons,

according to Mirza et al. (2025).[33] So 1-day ahead forecasting would be the most accurate choice for this study.

Since the objective of this study is not to maximize the performance of the models but to have a fair comparison, this study employed **static split** instead of **one-step rolling** forecast which is more widely used in real-world finance. The **one-step rolling** prediction would produce more accurate results as it retrains on the new data by giving each model different *training windows per step*, but it would end up using much more training time than usual. Additionally, **static split** emphasizes the *modeling approach* to compare LRs and RNNs rather than their financial performance.

3.5 Step 5: Evaluation Methods

The performance of the models were evaluated by using *thresholded* (0.5) metrics of *accuracy* - proportion of correctly classified predictions*, precision*- proportion of positive true predictions, *recall* - proportion of correct actual positives, and *F1 score* - average of precision and recall. *Probabilistic* metrics are also used such as *ROC-AUC* - distinguishing ability across thresholds, *PR-AUC* - precision-recall area under curve, *LogLoss* - penalizing wrong confident predictions, and *Brier score* - calibration quality. The higher the better ones are *accuracy, precision, recall, F1, ROC-AUC,* and *PR-ROC*. Conversely, the lower the better ones are *LogLoss* and *Brier score.*

[33] Mirza, Fuat Kaan, Önder Pekcan, Mustafa Hekimoğlu, and Tunçer Baykaş. "Stock Price Forecasting through Symbolic Dynamics and State Transition Graphs with a Convolutional Recurrent Neural Network Architecture." *Neural Computing and Applications*, vol. 37, 2025, pp. 15855–15890. https://doi.org/10.1007/s00521-025-11325-z.

4 Results and Analysis

The results of the experiment are analyzed below. Downward in this section, the analysis becomes more specific.

4.1 Aggregate performance analysis

Overall Means Across Stocks (LR vs LSTM)

Model	Accuracy	Precision	Recall	F1	ROC_AUC	PR_AUC	LogLoss	Brier
LR	0.483	0.484	0.436	0.423	0.472	0.514	0.727	0.255
LSTM	0.494	0.573	0.393	0.386	0.471	0.522	0.700	0.253

Table 2 : Comparison of LR and LSTM on aggregate basis

As shown in Table 2, on an aggregate basis, across all 10 stocks LSTM showed **minor superior** performance over LR. LSTM performs better for *accuracy* and *precision*, respectively: LSTM at 0.494 versus LR at 0.483 and LSTM at 0.573 versus LR at 0.484. But, *recall* and *F1 score* show the opposite by favoring LR over LSTM with respectively LR(0.436) versus LSTM(0.393) and LR(0.423) versus LSTM(0.386).

For probabilistic metrics, *ROC_AUC* and *Brier score* shows almost identical results for both models. But for *PR_AUC* and *LogLoss* metrics, LSTM shows slightly better performance with respectively LSTM(0.522) versus LR(0.514) and LSTM(0.700) versus LR(0.727) (the lower the better for *LogLoss*).

4.2 Per-stock and per-sector analysis

Wins Per Metric (count of stocks)

Metric	LR wins	LSTM wins	Number of stocks
Accuracy	4	6	10
Precision	4	6	10
Recall	7	3	10
F1	7	3	10
ROC_AUC	5	5	10
PR_AUC	5	5	10
LogLoss	4	6	10
Brier	4	6	10

Table 3 : Wins per metric table

Table 3 reveals that model performance varies for individual stocks: LSTM wins in accuracy and precision for 6 out of 10 stocks each, where LR wins in recall and F1 score for 7 out of 10 stocks each. Hence, this shows LSTM makes *more conservative predictions* with *higher* precision but with *lower* recall.

Means by Sector (LR vs LSTM)

Sector	Model	Accuracy	Precision	Recall	F1	ROC_AUC	PR_AUC	LogLoss	Brier
E-commerce	LR	0.475	0.446	0.359	0.334	0.468	0.515	0.703	0.255
E-commerce	LSTM	0.497	0.644	0.135	0.196	0.460	0.510	0.698	0.252
Finance	LR	0.470	0.494	0.375	0.381	0.495	0.538	0.702	0.254
Finance	LSTM	0.495	0.537	0.510	0.520	0.491	0.576	0.695	0.251
Healthcare	LR	0.523	0.477	0.299	0.362	0.462	0.468	0.832	0.259
Healthcare	LSTM	0.537	0.673	0.199	0.259	0.455	0.497	0.709	0.258
Oil & Gas	LR	0.490	0.521	0.652	0.558	0.521	0.558	0.694	0.250
Oil & Gas	LSTM	0.447	0.466	0.654	0.508	0.456	0.471	0.704	0.255
Technology	LR	0.457	0.482	0.497	0.481	0.415	0.489	0.704	0.255
Technology	LSTM	0.492	0.542	0.465	0.449	0.493	0.555	0.693	0.250

Table 4 : Comparison of LR and LSTM on each sector

According to Table 4, analysis per sector also reveals interesting underlying patterns. For example, in *Finance*, LSTM shows better precision(0.537 vs 0.494) and PR_AUC(0.576 vs 0.538). Interestingly, the *Healthcare* sector has the most considerable differences. In this sector, LR has higher accuracy(0.523 vs 0.537) and higher recall(0.299 vs 0.199), and LSTM shows significantly higher precision(0.673 vs 0.477). Conversely, in the *Oil & Gas* sector, LR performs much better than LSTM in most metrics, e.g. in ROC_AUC(0.521 vs 0.456).

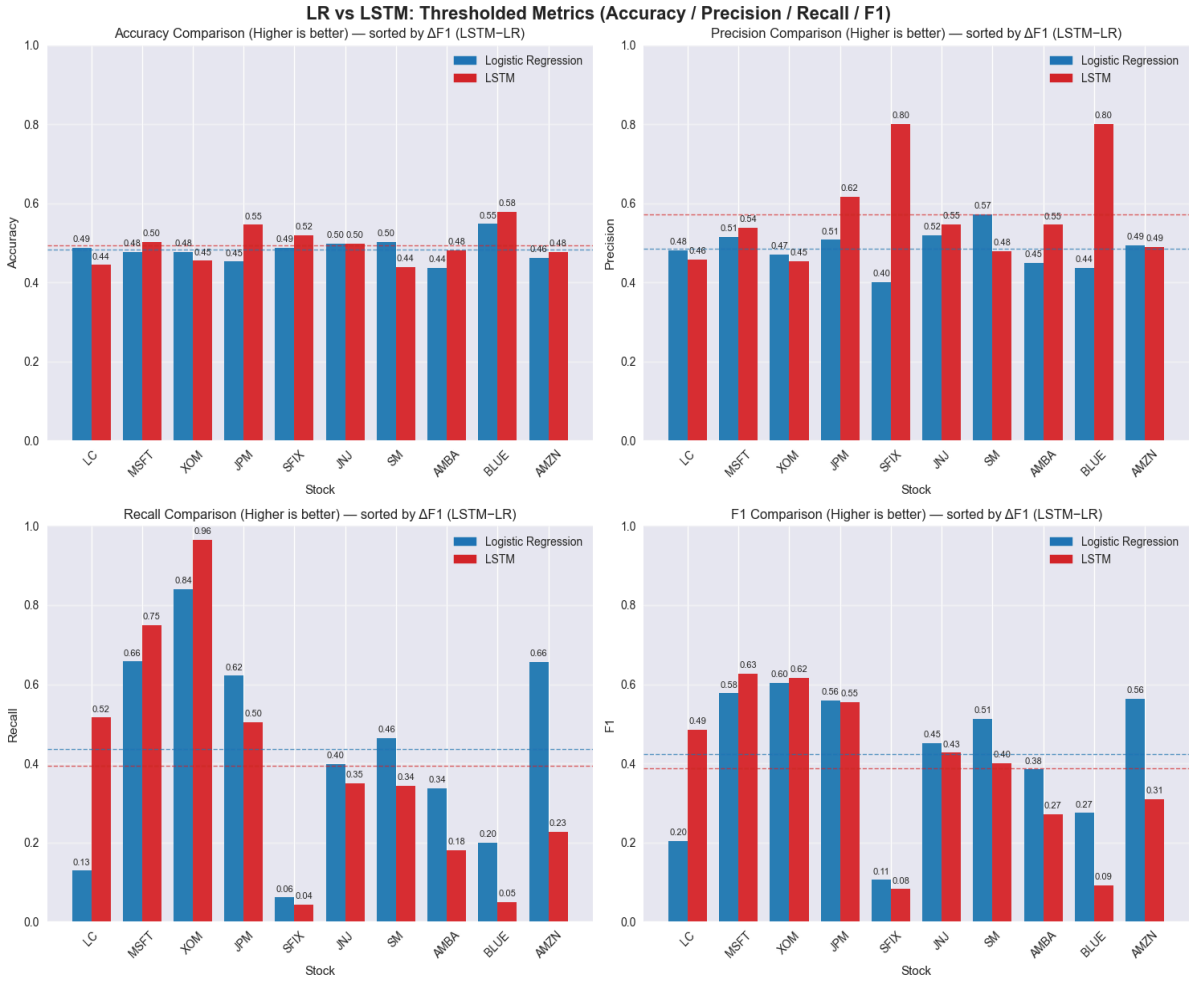


Figure 13 : Comparison of LR and LSTM by thresholded metrics on per-stock basis

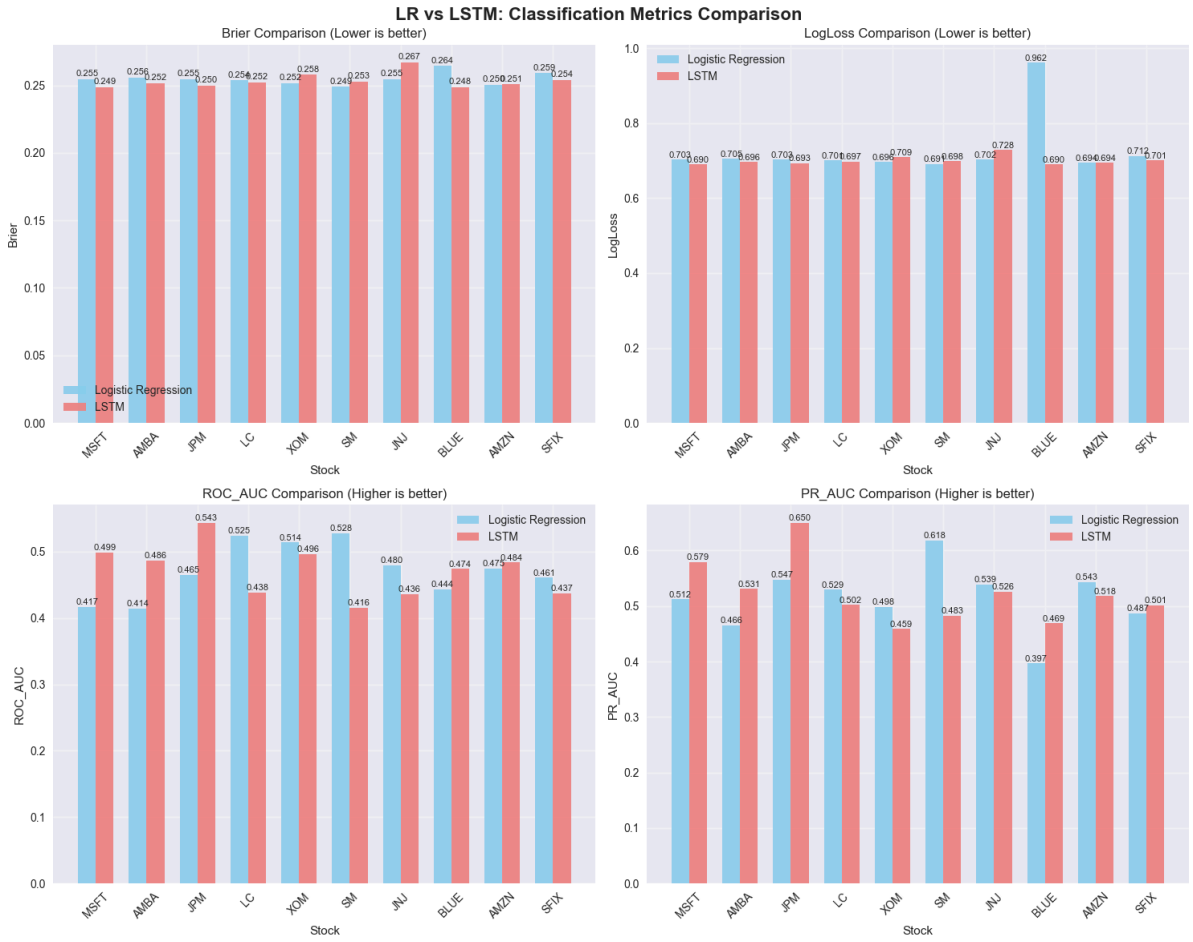


Figure 14 : Comparison of LR and LSTM by classification metrics on per-stock basis

In figure 13 and 14, other notable patterns are revealed through per-stock basis analysis, as it contains both large and small/mid cap companies for each sector. For instance, when considering MSFT, a stock of *large cap* Technology company, it is seen that across all 8 metrics LSTM shows **superiority**. Then when considered AMBA, a stock of *small/mid cap* Technology company, the similarities of superiority can be seen (except 2 metrics), but this time in **different magnitudes**. This observation suggests that the size of a company indirectly, through the amount of the noisyness of its stock data, affects the model performance, in which in this case LSTM had a chance to apply its complexity over a large cap company - MSFT.

4.3 LR Feature Importance analysis

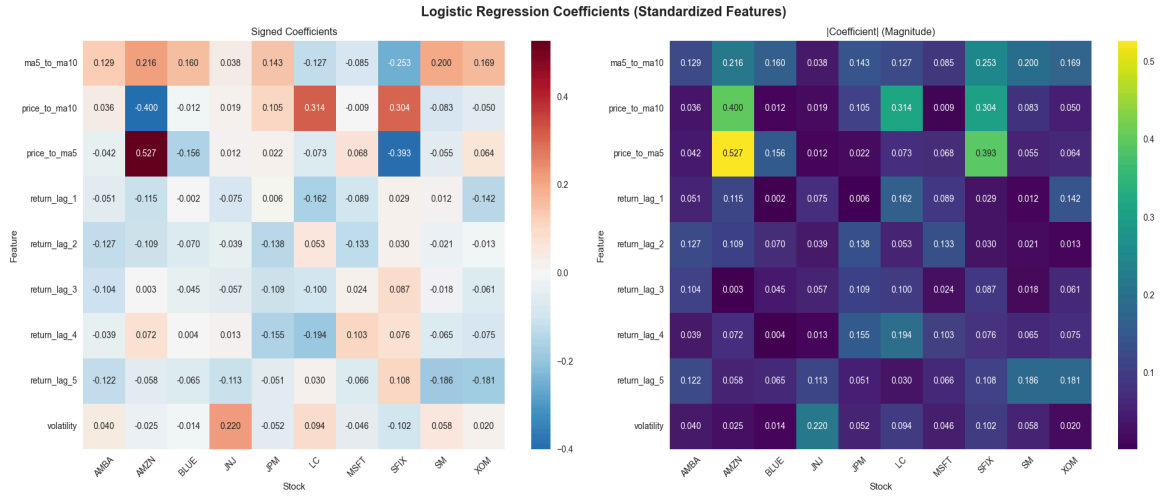


Figure 15 : The heat-map of logistic regression coefficients

The heat-map in figure 15 emphasizes the importance of *feature engineering* on the prediction of stock movements. First of all, each cell in this heat-map shows feature's coefficient for a stock with warmer or brighter meaning larger value of coefficient: positive means higher next day up odds and vice versa. With a heat-map it is easy to recognize features that matter most at a glance.

Moving averages (ma5_to_ma10, price_to_ma10, price_to_ma5) have very different signs and magnitudes, which means each feature has distinct prediction power. Additionally, it can be seen that *volatility* values are mostly positive which suggests that the odds of next day being up are higher.

Return lag features (from return_lag_1 to return_lag_5) show mixed signs, which suggest *momentum* effects in the market if coefficients are positive and *mean reversion* in the market if coefficients are negative.

Overall, LR's heat-map adds up to its *interpretability*, which helps us to recognize the importance of certain features and use them to match LSTM when the signal is simple and stable.

4.4 LSTM Training analysis

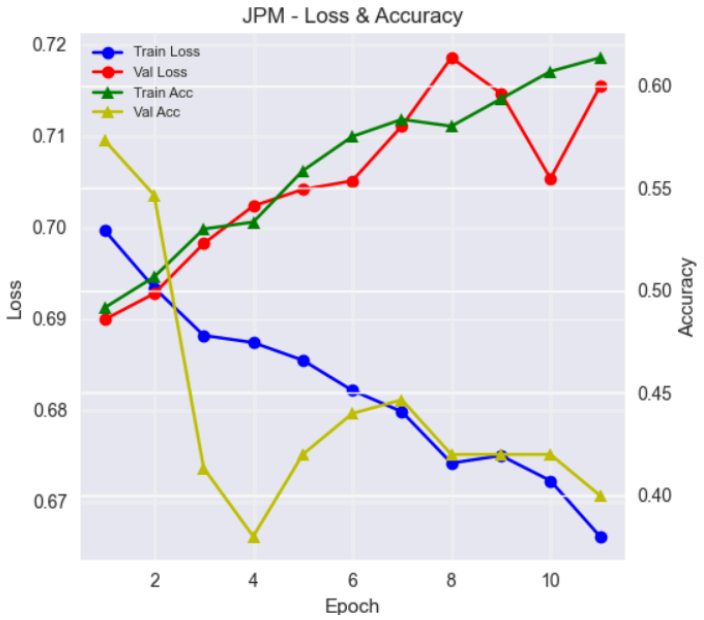


Figure 16 : The training curves of LSTM for stock JPM

Figure 17 : The training curves of LSTM for stock AMZN

In Figure 16 and Figure 17, the training curves for JPM and AMZN show that LSTM had *overfitting* and poor *generalization* during training. For JPM a critical *overfitting* happens when the training loss curve goes down but the validation loss curve increases after epoch 6. Moreover, as training accuracy increases but validation accuracy decreases for JPM, it means that LSTM poorly generalizes the patterns. Almost similar situation is happening for AMZN stocks, which again proves LSTM's *overfitting* and poor *generalization.*

Overall, these curves suggest that the complexity of LSTMs prevents them excelling at this relatively easy task of binary classification, instead it memorizes the complex patterns.

4.5 Computational Efficiency

Another major difference between LR and LSTM, *computational efficiency*, is proved by Figure 18:

Figure 18 : The aggregate training times of LSTM and LR

As shown LSTM took 6.4594 seconds for all stocks whereas LR only took 0.0063, roughly *1025 times longer*. When we consider the previous analysis, where we saw marginal

differences, this major difference clearly undermines the practicality of LSTM models for this study.

4.6 Considerations and Implications

When considered all together, the evidence does not claim a better model, rather it suggests the appropriate model for specific use cases. On average, LSTM had better precision and *PR_AUC* and lower *LogLoss* and *Brier*, but LR performed better with its *recall* and *F1 score* when it was drastically faster and more interpretable. This is what defines when to use them: if it is for "catching every UP day" or it needs to be easily debuggable with limited computational resources, then we need LR. Alternatively, if the aim is to have fewer but precise and confident signals from the stock market with no limitation on resources, then we need LSTM. This also can be observed further, as LSTM's learning curves showed overfitting in short-horizon equity data. Moreover, LR showed better results in stocks of *lower-volatility, steadier, larger-cap* companies in sectors of Finance, Healthcare, and Oil & Gas. In contrast, LSTM was more effective in *more volatile markets*, usually Tech and E-commerce and *smaller caps*, where patterns are relatively complex.

5 Conclusion

For the research question, *"To what extent are Recurrent Neural Networks more effective than traditional Logistic Regression models in predicting stock price movements?",* it can be concluded, based on empirical evidence, that Recurrent Neural Networks (LSTMs) are *only conditionally* more effective than Logistic Regression models in predicting stock price movements.

LSTMs had better results on more volatile stocks of certain sectors, especially smaller caps. Nevertheless, LSTMs did not fully outperform LR models. Logistic regression had gains on some metrics and was more interpretable and faster with limited resources. These results pointed to the best use-case scenarios of both models.

However as these conclusions are made on specific dataset, model design, period, and horizon, the findings *should not be generalized* or *extended to all situations* without further research. In conclusion, the research question was effectively answered, but still bringing up more questions on this topic.

6 Limitations and further research

In the conduct of this study, as mentioned earlier, there have been limitations affecting the implementation and results. Firstly, for stock price movements, due to limited sources of stock data, *OHCLV* values were customized to mimic the *Adjusted Prices*, which are better data values to use. Even though they are better than *OHCLV* values, the study still lacks the ability to take market influences, such as government policies, market sentiment, or natural disasters. Another improvement could be done by increasing the magnitude of stock data and prediction time and horizon. Secondly, the *static split* of 80/20 could be replaced by a *one-step rolling forecast* for better financial performance of models.

Lastly, as mentioned in the Technical foundations section, the code developed for experiment could be further improved by advanced *hyperparameters tuning* and *feature engineering*, which could have crucial improvements in the performance.

However, all these improvements were out of the scope of this study which does not aim to have the most optimized performance, but rather fair comparison between models. In spite of limitations, the study still has considerable and clarifying results. In university, I intend to extend this research with larger resources and more expertise.

7 Works Cited

"API Documentation." *Alpha Vantage*, www.alphavantage.co/documentation/.

Bischl, Bernd, et al. "Hyperparameter Optimization: Foundations, Algorithms, Best Practices and Open Challenges." *arXiv*, 24 Nov. 2021, doi.org/10.48550/arXiv.2107.05847 .

Cahuantzi, Roberto, Xinye Chen, and Stefan Güttel. "A Comparison of LSTM and GRU Networks for Learning Symbolic Sequences." *arXiv*, 5 July 2021, arxiv.org/abs/2107.02248 .

Das, Saurabh, et al. "Trustworthy Artificial Intelligence in Finance: Explainability, Fairness, and Related Principles for AI in Financial Services." *arXiv*, 16 May 2025, arxiv.org/abs/2505.24650 .

"Difference Between Model Parameters vs Hyperparameters." *GeeksforGeeks*, 2025, www.geeksforgeeks.org/machine-learning/difference-between-model-parameters-vs-hyperparameters/.

Fischer, Thomas, and Christopher Krauss. *Deep Learning with Long Short-Term Memory Networks for Financial Market Predictions. FAU Discussion Papers in Economics*, no. 11, Friedrich-Alexander University Erlangen-Nürnberg, 2017. *ECONIS – Online Catalogue of the ZBW*, https://www.econbiz.de/Record/deep-learning-with-long-short-term-memory-networks-for-financial-market-predictions-fischer-thomas/10011644167.

Frost, Jim. "Using Moving Averages to Smooth Time Series Data." *Statistics by Jim*, statisticsbyjim.com/time-series/moving-averages-smoothing/ .

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. *Deep Learning.* MIT Press, 2016.

Hastie, Trevor, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction.* 2nd ed., Springer, 2009.

"Introduction to Non-Stationary Processes." *Investopedia*, 5 Jan. 2022, www.investopedia.com/articles/trading/07/stationary.asp.

Kalita, Debasish. "A Brief Overview of Recurrent Neural Networks (RNN)." *Analytics Vidhya*, 7 Nov. 2023, www.analyticsvidhya.com/blog/2022/03/a-brief-overview-of-recurrent-neural-networks-rnn/.

Kenton, Will. "What Are the Key Factors That Cause the Market to Go Up and Down?" *Investopedia*, Dotdash Meredith, 21 Sept. 2023, www.investopedia.com/ask/answers/100314/what-are-key-factors-cause-market-go-and-down.asp.

"Logistic Regression in Machine Learning." *GeeksforGeeks*, 2 Aug. 2025, www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/.

Makridakis, S., E. Spiliotis, and V. Assimakopoulos. "Statistical and Machine Learning Forecasting Methods: Concerns and Ways Forward." *PLoS ONE*, vol. 13, no. 3, 2018, e0194889. https://doi.org/10.1371/journal.pone.0194889.

Manchanda, Chitwan. "Logistic Regression in Machine Learning (from Scratch!!)." *Medium*, 11 Mar. 2022, medium.com/aiguys/logistic-regression-in-machine-learning-from-scratch-62f45048c571 .

Mirza, Fuat Kaan, Önder Pekcan, Mustafa Hekimoğlu, and Tunçer Baykaş. “Stock Price Forecasting through Symbolic Dynamics and State Transition Graphs with a Convolutional Recurrent Neural Network Architecture.” *Neural Computing and Applications*, vol. 37, 2025, pp. 15855–15890. doi.org/10.1007/s00521-025-11325-z .

Nyuytiymbiy, Kizito. “Parameters, Hyperparameters, Machine Learning.” *Towards Data Science*, 7 Mar. 2023, towardsdatascience.com/parameters-and-hyperparameters-aa609601a9ac .

O’Malley, Tom, et al. *KerasTuner. Keras*, 2019, keras.io/keras_tuner/ .

Olah, Christopher. “Understanding LSTM Networks.” *Colah’s Blog*, 27 Aug. 2015, colah.github.io/posts/2015-08-Understanding-LSTMs/ .

Pandey, Rajat. *Stock Price Prediction: An Integrated Review of Traditional and Modern Computational Models. SSRN*, 2025, https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5353725.

RITHB. “Logistic Regression and Regularization: Avoiding Overfitting and Improving Generalization.” *Medium*, 5 Jan. 2023, medium.com/@rithpansanga/logistic-regression-and-regularization-avoiding-overfitting-and-improving-generalization-e9afdcddd09d .

“RNN vs LSTM vs GRU vs Transformers.” *GeeksforGeeks*, 23 July 2025, www.geeksforgeeks.org/deep-learning/rnn-vs-lstm-vs-gru-vs-transformers/.

“Time Series Analysis and Forecasting.” *GeeksforGeeks*, 23 July 2025, www.geeksforgeeks.org/machine-learning/time-series-analysis-and-forecasting/#what-is-time-series-forecasting.

"Time Series Forecasting - Meaning." *Polestar Analytics*, www.polestarllp.com/glossary/time-series-forecasting.

"What is LSTM - Long Short Term Memory?" *GeeksforGeeks*, 26 July 2025, www.geeksforgeeks.org/deep-learning/deep-learning-introduction-to-long-short-term-memory/.

"Yfinance." *PyPI*, 21 Jan. 2024, pypi.org/project/yfinance/ .

"Yahoo Finance - Stock Market Live, Quotes, Business and Finance News." *Yahoo Finance*, finance.yahoo.com .

Zaidi, Makram, and Amina Amirat. "Forecasting Stock Market Trends by Logistic Regression and Neural Networks: Evidence from KSA Stock Market." *International Journal of Economics, Commerce and Management*, vol. 4, no. 6, June 2016, pp. 220–234. ijecm.co.uk/wp-content/uploads/2016/06/4614.pdf .